



Indian Institute of Information Technology
Sri City, Chittoor

Searching & Hashing Algorithms

Searching – Basic Concepts

- A set of values in an array or list
- A search key to be found
- To check whether the key exists in the array or not
- If it exists in the array, then find its index
- If not return an error (unsuccessful search)
- Basic Searching Techniques:
 - Linear Search or Sequential Search
 - Binary Search

Linear Search

- Only method for unsorted data
- Scan the array or list from start to end
- For a searching algorithm S:
 - **Input:** A list of items $[a_1, a_2, a_3, \dots, a_n]$, and a search key k
 - **Output:**
 - *A Boolean (True/False) value:* Indicates whether the element was found or not
 - *The position:* At which the element was found

Linear Search

- Assumptions:
 - The list indexing is from L to U , where $0 \leq L \leq U$
 - The *LINEAR_SEARCH* function takes two arguments: a *list* $A[L..U]$ and the *search key*
 - The *LINEAR_SEARCH* function returns
 - The position of the element, if the element is found
 - -1, if the element is not found (will this work if assumption 1 is not enforced?)

Linear Search – Algorithm & Analysis

LINEAR_SEARCH (*A*, *key*)

i = *L*

while *i* < *U*, **do**

if *A*[*i*] == *key*

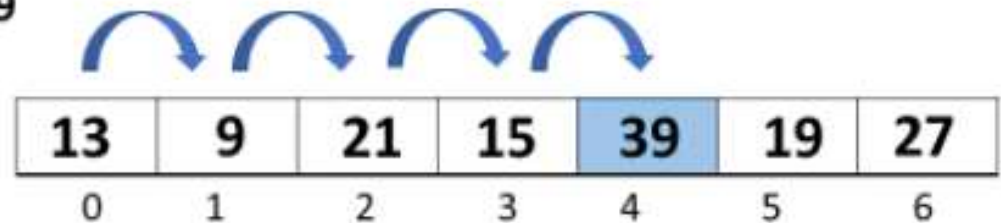
return *i*

return -1

■ **Example:**

Searched Element

39



- Time Complexity of searching algorithms depends on the number of comparisons required to find the search key

Linear Search – Running Time

- **Key is NOT in array-Unsuccessful:** # Comparisons= n
- **Key is in array - Successful**
 - #Comparisons(min) = 1, if key is first element
 - #Comparisons(max) = n , if key is last element
- **Best Case:**
 - $key == A[L]$
 - Complexity: $O(1)$
- **Worst Case:**
 - $key == A[U-1]$ or element is absent
 - Complexity: $O(n)$ [Assuming that $L - U + 1 = n$]
- **Average Case:**
 - Complexity: $O(n)$ [How?]

Linear Search – Applications

- Linear search can be applied to both 1D and multi-dimensional arrays
- Linear search is easy to implement
- Effective when the array contains only a few elements
- Linear Search is also efficient when the search is performed to fetch a single search in an unordered-List

Binary Search

- If the input array is sorted, then we can reduce the complexity of searching
 - Binary Search

Steps in Binary Search Algorithm

- Compare *search key k* with the middle element in an array
 - If *k* matches with the middle element, we return the *mid index*
 - Else If *k* is greater than the mid element, then *k* can only lie in the right half sub-array after the mid element. So, we recur for the right half
 - Else (*k* is smaller) recur for the left half

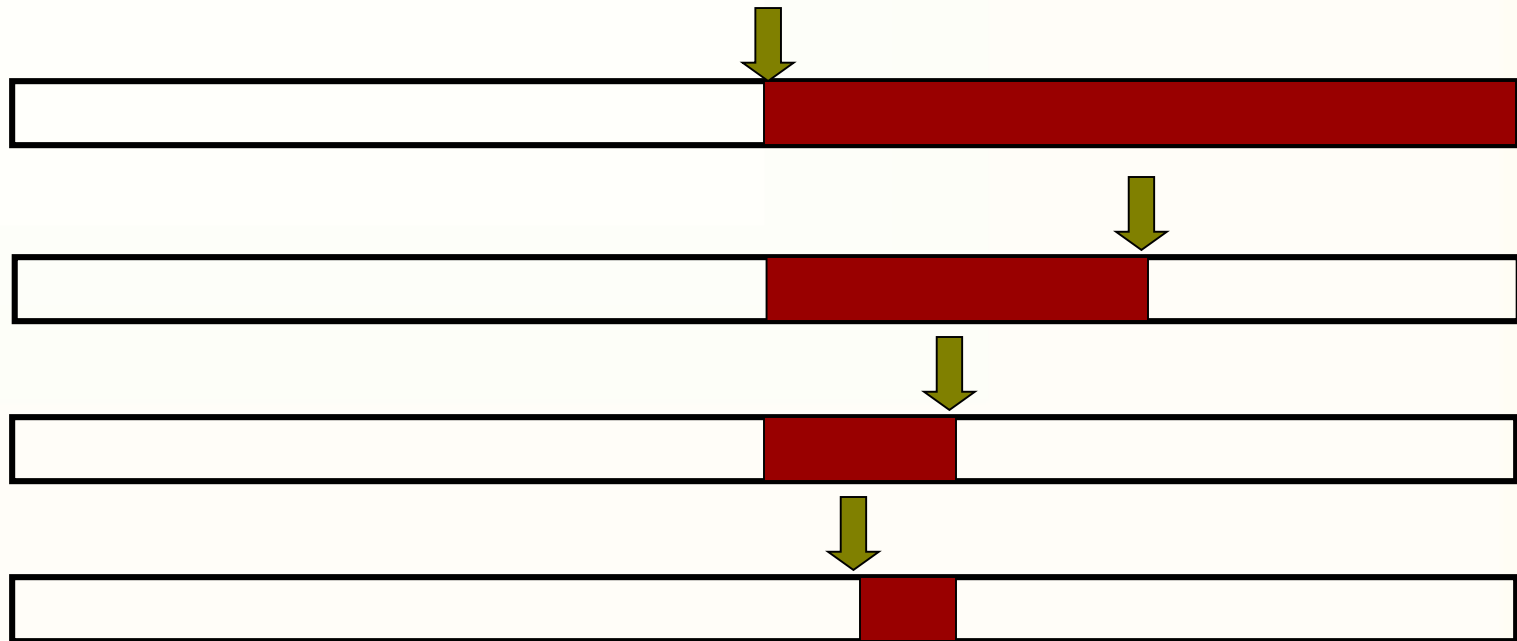
Binary Search- Example

- The array contains 10 elements in sorted order
- Search key =23

Searching	Search 23	0	1	2	3	4	5	6	7	8	9
		2	5	8	12	16	23	38	56	72	91
	23 > 16 take 2 nd half	L=0	1	2	3	M=4	5	6	7	8	H=9
		2	5	8	12	16	23	38	56	72	91
	23 < 56 take 1 st half	0	1	2	3	4	L=5	6	M=7	8	H=9
		2	5	8	12	16	23	38	56	72	91
	Found 23, Return 5	0	1	2	3	4	L=5, M=5	H=6	7	8	9
		2	5	8	12	16	23	38	56	72	91

Binary Search for Key- Case Study

Given a sorted array of values, find a key



Searching

Binary Search - Algorithm

BINSEARCH (Arr, beg, end, key)
 while *beg* ≤ *end*, ***do***
 mid = (*beg* + *end*) / 2
 if *Arr*[*mid*] == *key*
 return *mid*
 else if *Arr*[*mid*] > *key*
 end = *mid* - 1
 else
 beg = *mid* + 1
 return -1

NOTE: The function will be called as BINSEARCH(Arr, L, U, key), where L and U are the first and last valid indices of the array

Binary Search-Complexity Analysis

- If the *search key* is the *middle element* of the list then **one comparison** is required to search the key
- So, **Best Case Time Complexity** of Binary Search: $O(1)$
- **Worst Case Time Complexity** of Binary Search: $O(\log n)$
- For Binary Search,
$$T(n) = T(n/2) + O(1) \text{ // } O(1) \text{ for single comparison}$$
- Apply Master Theorem
Compare with $T(n) = aT(n/b) + n^k$
Here, $a = 1$, $b = 2$, $k=0$ so, $a=b^k$
- Then $T(n) = O(n^k \log(n)) \text{ // } k = 0$
- So, $T(n) = O(n^0 \log(n)) = O(\log n)$

Binary Search-Analysis

- Let the Binary Search terminates after k iterations
 - At each iteration, the array is ***divided by half***
 - At ***iteration 1***, length of array = n
 - At iteration 2, Length of array = $n/2$
 - At iteration 3, Length of array = $(n/2)/2 = (n/2^2)$
 - Therefore, ***after Iteration k*** , Length of array = $n/2^k$
 - Also, we know that after k divisions,
the length of the array becomes 1
 - Therefore Length of array = $(n/2^k) = 1 \Rightarrow n = 2^k$
- Applying log function on both sides: $\log(n) = \log(2^k)$
 $\Rightarrow \log(n) = k \log(2)$, Therefore, $k = \log_2(n)$
- Hence the worst case time complexity is $(\log n)$

Binary Search-Analysis & Applications

- **Average Case Time Complexity** of Binary Search:
 $O(\log n)$

Some of the real life Applications:

1. Dictionary
2. Debugging a linear piece of code

Important Points

❑ Linear Search is simplest and least efficient algorithm or searching.

❑ **Linear Search is $O(N)$ operation**

❑ Binary Search has the best possible efficiency for a search algorithm

❑ **Binary Search is $O(\log N)$**



THANK YOU